

JobAgent Enterprise

Technical Documentation

Version 1.0.0 | Generated: June 29, 2026

JobAgent Enterprise - Complete Technical Documentation

Version: 1.0.0

Last Updated: June 29, 2026

Document Type: Technical Reference Guide

Audience: Developers, DevOps Engineers, System Architects

Table of Contents

- Project Overview
- Technology Stack
- System Architecture
- Folder Structure
- API Documentation
- Environment Variables & Configuration
- Database Schema
- Authentication & Authorization
- Frontend Architecture
- Backend Architecture
- N8N Workflows
- Dependencies
- Project Resource Mapping
- Application Flow
- Security & Deployment

1. Project Overview

1.1 Project Purpose

JobAgent Enterprise is a comprehensive AI-powered career assistant platform that combines automated job scraping, intelligent resume analysis, interview preparation, and career insights into a unified system. The platform helps job seekers optimize their career journey through:

- **Multi-Platform Job Scraping:** Automated collection from Naukri, RemoteOK, and Wellfound
- **AI-Powered Resume Analysis:** Advanced ATS (Applicant Tracking System) scoring with 10 weighted components
- **Interview Preparation:** Role-specific questions with AI-generated tips and sample answers
- **Career Insights:** Market data, salary ranges, and growth projections
- **Skill Gap Analysis:** Identifies missing skills and provides learning roadmaps
- **Job Recommendations:** AI-driven matching based on user profiles
- **Feedback Processing:** Intelligent sentiment analysis and prioritization

1.2 Key Features

Feature	Description	Technology
Job Scraping	Multi-platform job collection with pagination	BeautifulSoup, Selenium, Playwright
Resume Analysis	100-point ATS scoring system	Rule-based engine + AI enhancement
Interview Prep	500+ role-specific questions	Static database + AI generation
Career Insights	Market trends and salary data	Internal DB + AI analysis
User Management	Authentication, profiles, preferences	Flask-Login, SQLAlchemy
Automation	Background job recommendations	N8N workflows
Feedback System	Sentiment analysis and routing	AI-powered processing

1.3 Project Goals

- **Primary:** Help job seekers optimize their resumes and prepare for interviews
 - **Secondary:** Aggregate job listings from multiple platforms
 - **Tertiary:** Provide career guidance and skill development roadmaps
-

2. Technology Stack

2.1 Backend Technologies

Technology	Version	Purpose
Python	3.11+	Core programming language
Flask	3.0.0	Web framework and API server
Flask-Login	0.6.3	Session management and authentication
Flask-SQLAlchemy	3.1.1	ORM for database operations
SQLAlchemy	2.0.23	Database abstraction layer
PostgreSQL	15	Primary database (production)
SQLite	-	Development database
Redis	7	Caching and session storage

2.2 Frontend Technologies

Technology	Version	Purpose
HTML5	-	Template rendering
CSS3	-	Styling and animations
JavaScript (ES6+)	-	Client-side interactivity
Jinja2	-	Template engine for Flask

2.3 AI/ML Technologies

Technology	Version	Purpose
Google Gemini	0.3.2	AI-powered resume analysis, interview questions, career insights
OpenAI	1.6.1	Optional enhanced features
Hugging Face	0.19.4	Resume classification and analysis

2.4 Scraping Technologies

Technology	Version	Purpose
BeautifulSoup4	4.12.2	HTML parsing for Naukri
lxml	4.9.3	XML/HTML parser backend
Selenium	-	Dynamic content scraping
Playwright	-	Wellfound API interception
Requests	2.31.0	HTTP client for API calls
httpx	0.25.2	Async HTTP client
aiohttp	3.9.1	Async HTTP framework

2.5 Automation & Integration

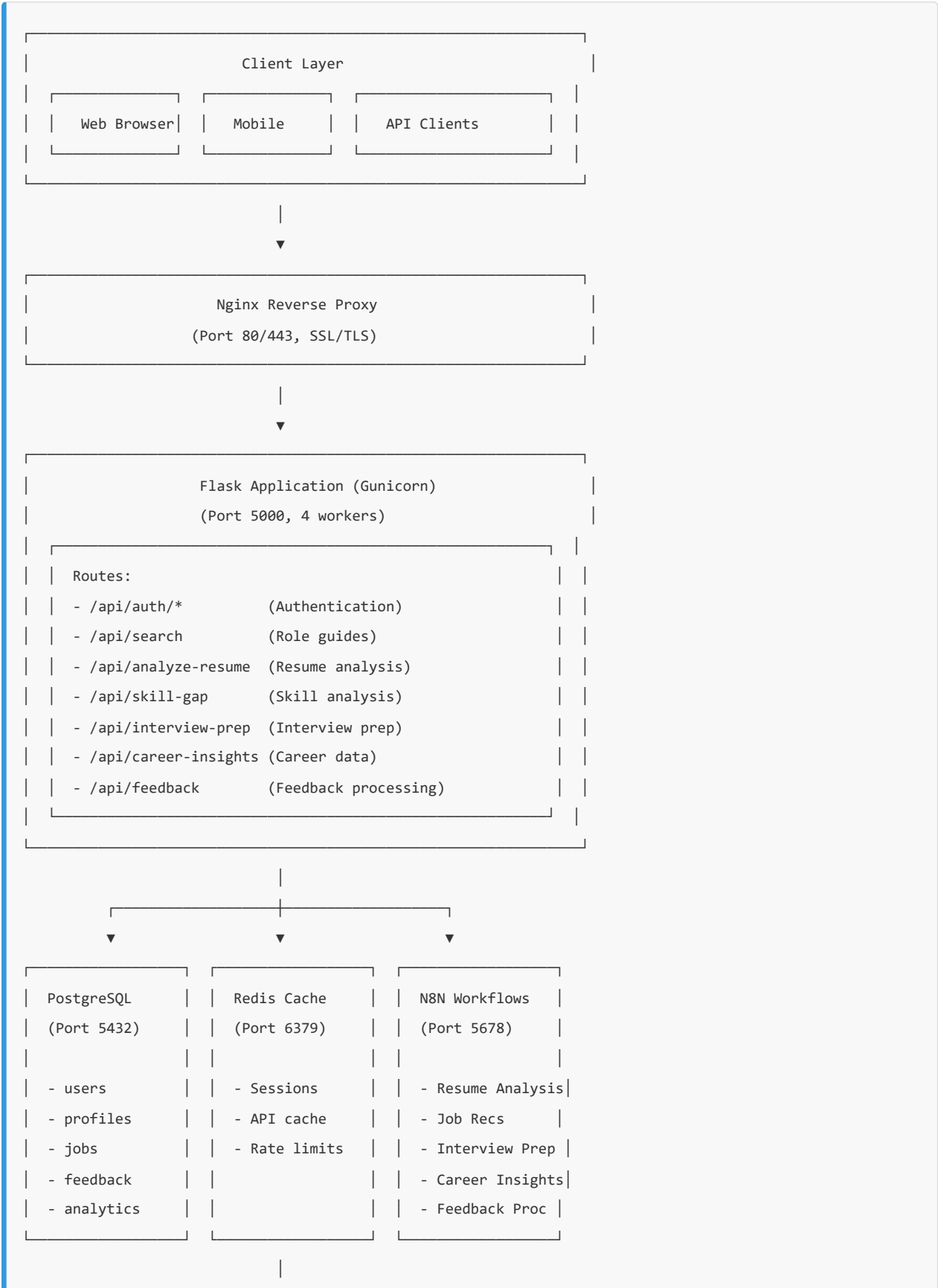
Technology	Version	Purpose
N8N	Latest	Workflow automation
Firecrawl	0.0.16	Web scraping (Wellfound)

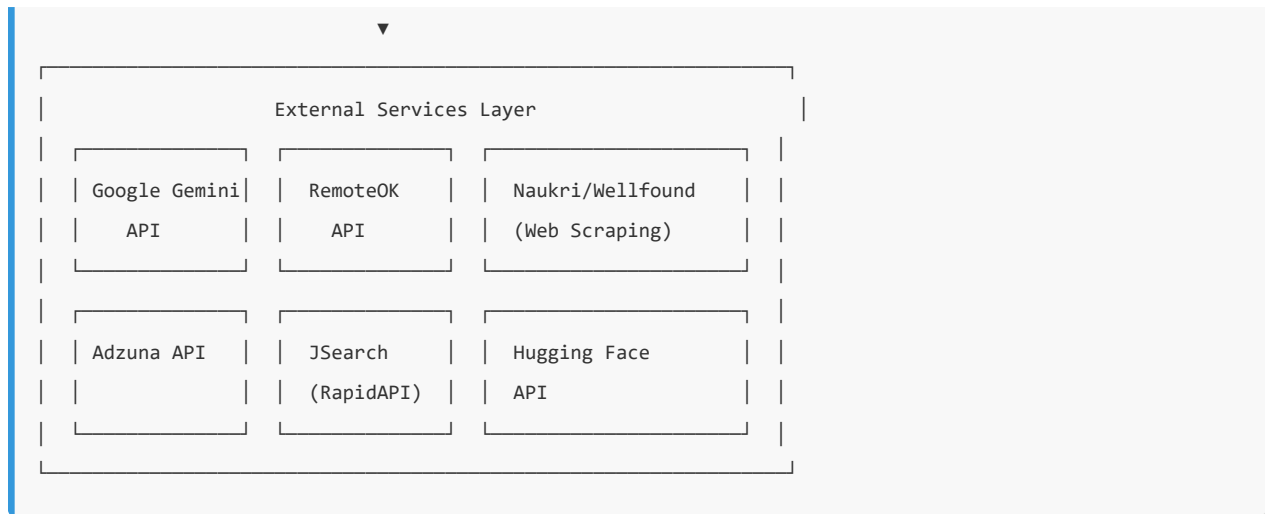
2.6 DevOps & Deployment

Technology	Version	Purpose
Docker	-	Containerization
Docker Compose	-	Multi-container orchestration
Gunicorn	-	Production WSGI server
Nginx	Alpine	Reverse proxy and load balancer
Prometheus	Latest	Metrics collection
Grafana	Latest	Monitoring dashboards

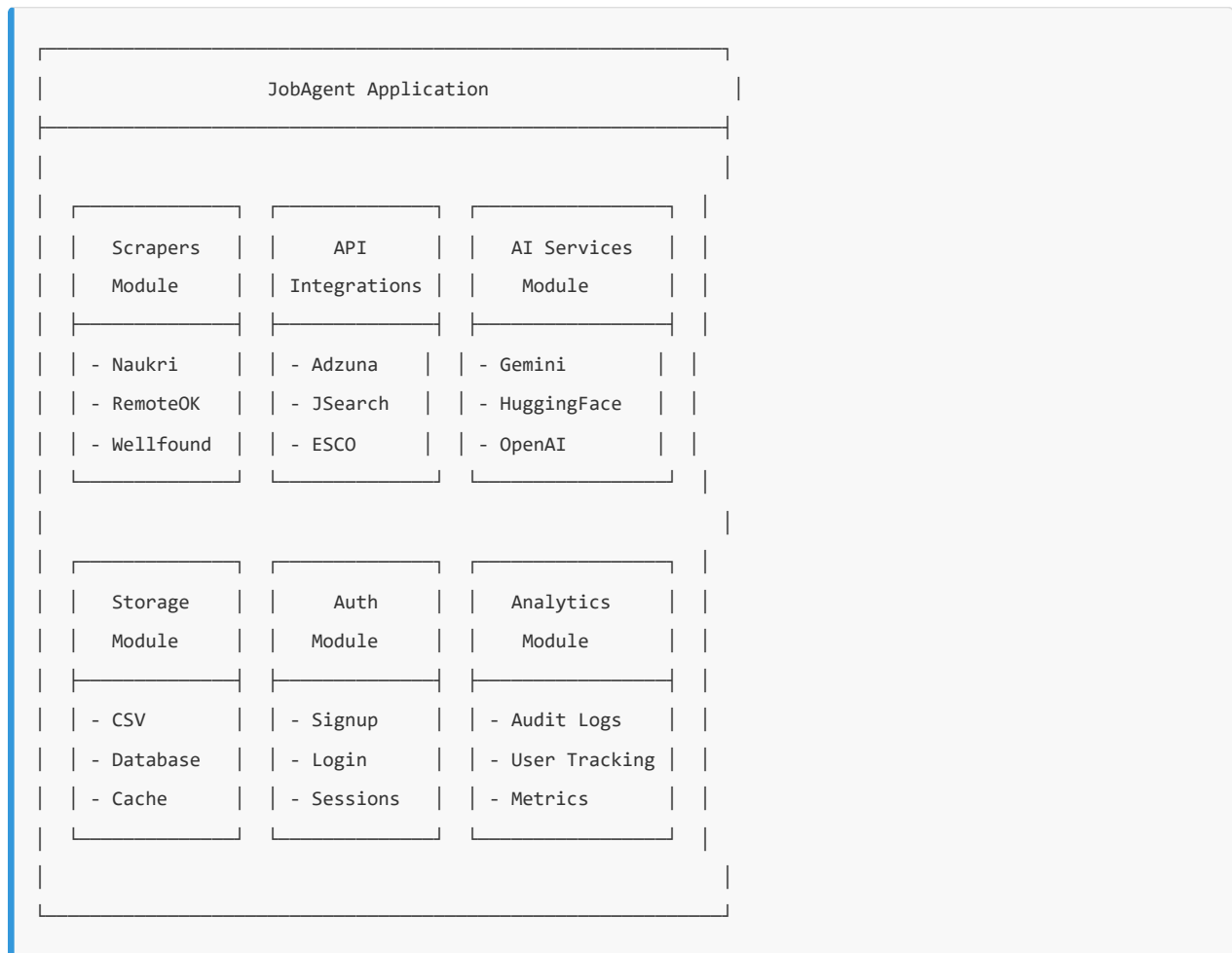
3. System Architecture

3.1 High-Level Architecture





3.2 Component Architecture



3.3 Data Flow Architecture



4. Folder Structure

4.1 Complete Directory Tree

```
job-agent/
├─ app.py                # Main Flask application (2493 lines)
├─ main.py              # CLI entry point for job scraping
├─ config.py            # Configuration management
├─ requirements.txt      # Python dependencies
├─ Dockerfile           # Production Docker image
├─ docker-compose.yml   # Multi-container orchestration
├─ .env.example         # Environment variable template
├─ .gitignore           # Git ignore rules
├─ README.md            # Project documentation
├─
├─ models/              # Database Models
│  └─ __init__.py
│  └─ user_model.py     # User, Profile, Jobs, Applications (366 lines)
│  └─ job_model.py      # Job data models and validation
│  └─
├─ scrapers/            # Job Board Scrapers
│  └─ __init__.py
│  └─ base_scraper.py   # Abstract base class (285 lines)
│  └─ naukri_scraper.py # Naukri.com scraper (400 lines)
│  └─ remoteok_scraper.py # RemoteOK API scraper (297 lines)
│  └─ wellfound_scraper.py # Wellfound Playwright scraper (147 lines)
│  └─
├─ utils/               # Utility Modules
│  └─ __init__.py
│  └─ auth.py           # Authentication utilities (380 lines)
│  └─ api_integrations.py # Free API integrations (384 lines)
│  └─ logger.py         # Logging configuration
│  └─ query_parser.py   # Search query parsing
│  └─ html_cache.py     # HTML caching for debugging
│  └─
├─ storage/             # Data Storage
│  └─ __init__.py
│  └─ csv_storage.py    # CSV file handler
│  └─
├─ templates/           # HTML Templates
│  └─ index.html        # Main application page
│  └─ login.html        # Login page
│  └─ signup.html       # Registration page
│  └─ forgot_password.html # Password reset request
│  └─ reset_password.html # Password reset form
├─
└─
```

```
├─ static/                                # Static Assets
│   └─ css/
│       └─ style.css                      # Main stylesheet
│       └─ js/
│           └─ app.js                     # Frontend JavaScript
│
├─ n8n/                                    # N8N Workflows
│   └─ workflows/
│       ├── resume_analysis_workflow.json  # AI resume analysis
│       ├── job_recommendations_workflow.json # Daily job matching
│       ├── interview_prep_workflow.json   # Question generation
│       ├── career_insights_workflow.json  # Market insights
│       └─ feedback_processing_workflow.json # Feedback analysis
│
├─ docs/                                  # Documentation
│   ├── architecture.md
│   ├── context.md
│   ├── csv_storage_guide.md
│   ├── DEPLOYMENT_GUIDE.md
│   ├── DOCUMENTATION.html
│   ├── ENTERPRISE_ARCHITECTURE.md
│   ├── ENTERPRISE_UPGRADE_COMPLETE.md
│   ├── IMPROVEMENTS_REPORT.md
│   ├── n8n_workflows.md
│   ├── naukri_scraper.md
│   ├── remoteok_scraper.md
│   ├── setup.md
│   └─ wellfound_scraper.md
│
├─ tests/                                  # Test Suite
│   ├── __init__.py
│   ├── test_naukri_scraper.py
│   ├── test_remoteok_scraper.py
│   └─ test_wellfound_scraper.py
│
├─ instance/                              # Flask Instance
│   └─ jobagent.db                        # SQLite database (development)
│
├─ job-platform/                          # Legacy/Testing Platform
│   ├── app.py
│   ├── config.py
│   ├── database.py
│   ├── test_api.py
│   └─ ...
│
└─ farsan-showcase/                      # Separate showcase project
    └─ package.json
```

```
├─ next.config.ts
└─ src/
```

4.2 Core Files Explanation

4.2.1 app.py (2493 lines)

Purpose: Main Flask application with comprehensive career assistance features

Key Components:

- **Role Expansion Database:** Maps 13 core roles to 20+ related job titles
- **Skills Database:** 12 categories with 200+ skills for matching and gap analysis
- **Role Guides:** Detailed guides for 13 roles with responsibilities, skills, salaries, interview topics
- **Resume Analysis Engine:** 10-component weighted scoring system (100 points total)
- **Interview Questions:** 500+ questions across 8 role categories
- **Career Insights:** Market data for 6 primary roles
- **Learning Resources:** 30+ technologies with learning paths
- **Flask Routes:** 15+ API endpoints for authentication, analysis, and dashboard

Dependencies: Flask, Flask-Login, SQLAlchemy, models.user_model, config

4.2.2 main.py (111 lines)

Purpose: CLI entry point for standalone job scraping

Functionality:

- Accepts command-line arguments for job role, location, output file, limit
- Orchestrates scraping from Naukri, RemoteOK, and Wellfound
- Filters jobs by role relevance
- Saves results to CSV

Usage:

```
python main.py --job-role "Software Engineer" --location "Bangalore" --output jobs.csv --limit 50
```

4.2.3 config.py (55 lines)

Purpose: Centralized configuration management

Configuration Items:

- Firecrawl API key
- Scraping parameters (limit, timeout, retry attempts)
- Rate limiting delay
- Logging configuration
- Storage settings

Pattern: Dataclass with environment variable loading

5. API Documentation

5.1 Internal REST APIs

5.1.1 Authentication APIs

POST /api/auth/signup

Purpose: Register a new user account

Request Body:

```
{
  "email": "user@example.com",
  "username": "johndoe",
  "password": "SecurePass123!",
  "confirm_password": "SecurePass123!"
}
```

Response (Success - 201):

```
{
  "success": true,
  "message": "Account created successfully! Welcome to JobAgent.",
  "user": {
    "id": 1,
    "email": "user@example.com",
    "username": "johndoe",
    "is_verified": true,
    "created_at": "2026-06-29T10:30:00",
    "last_login": null
  }
}
```

Response (Error - 400):

```
{
  "success": false,
  "message": "All fields are required."
}
```

Validation Rules:

- Email: Valid format, checked for duplicates
- Username: 3+ characters, alphanumeric + underscores only
- Password: 8-128 chars, must include uppercase, lowercase, number, special character
- Password confirmation must match

Used By: `templates/signup.html` , `static/js/app.js`

POST /api/auth/login

Purpose: Authenticate user and create session

Request Body:

```
{
  "email": "user@example.com",
  "password": "SecurePass123!",
  "remember": false
}
```

Response (Success - 200):

```
{
  "success": true,
  "message": "Login successful!",
  "user": {
    "id": 1,
    "email": "user@example.com",
    "username": "johndoe",
    "is_verified": true,
    "created_at": "2026-06-29T10:30:00",
    "last_login": "2026-06-29T10:30:00"
  }
}
```

Response (Error - 401):

```
{
  "success": false,
  "message": "Invalid password. 4 attempt(s) remaining before account is locked."
}
```

Security Features:

- Account lockout after 5 failed attempts (15-minute lock)
- Session token generation
- Audit logging
- IP address and user agent tracking

Used By: `templates/login.html` , `static/js/app.js`

POST /api/auth/logout

Purpose: Log out current user

Headers: Requires authentication cookie

Response (Success - 200):

```
{
  "success": true,
  "message": "Logged out successfully."
}
```

Actions:

- Deactivates all user sessions
 - Clears Flask session
 - Creates audit log entry
-

POST /api/auth/forgot-password

Purpose: Initiate password reset flow

Request Body:

```
{
  "email": "user@example.com"
}
```

Response (Success - 200):

```
{
  "success": true,
  "message": "If an account exists with this email, you will receive a password reset link."
}
```

Security Note: Does not reveal whether email exists in system

Actions:

- Invalidates old reset tokens
 - Generates new secure token (48 chars, URL-safe)
 - Token expires in 1 hour
 - Logs token (in production, would send email)
-

POST /api/auth/reset-password

Purpose: Complete password reset with token

Request Body:

```
{
  "token": "abc123...",
  "password": "NewSecurePass123!",
  "confirm_password": "NewSecurePass123!"
}
```

Response (Success - 200):

```
{
  "success": true,
  "message": "Password has been reset successfully. You can now log in with your new password."
}
```

Actions:

- Validates token and expiration
 - Updates password hash
 - Invalidates all active sessions
 - Resets login attempt counter
-

GET /api/auth/me

Purpose: Get current authenticated user data

Headers: Requires authentication

Response (Success - 200):

```
{
  "user": {...},
  "profile": {...},
  "stats": {
    "total_saved": 5,
    "total_applications": 3,
    "total_interviews": 2,
    "active_applications": 2,
    "offers": 1
  },
  "saved_jobs": [...],
  "applications": [...],
  "interviews": [...]
}
```

GET /api/auth/check

Purpose: Check if user is authenticated (for frontend)

Response (Authenticated - 200):

```
{
  "authenticated": true,
  "user": {
    "id": 1,
    "email": "user@example.com",
    "username": "johndoe"
  }
}
```

Response (Not Authenticated - 200):

```
{
  "authenticated": false
}
```

5.1.2 Role Guide APIs

POST /api/search

Purpose: Search for role guides and get related roles

Request Body:

```
{
  "query": "Python Developer",
  "roles": ["Python Developer", "Backend Developer", "Django Developer"]
}
```

Response (Success - 200):

```
{
  "query": "Python Developer",
  "roles": ["Python Developer", "Backend Developer", ...],
  "total": 20,
  "primary_guide": {
    "role_title": "Python Developer",
    "overview": "A Python Developer builds...",
    "responsibilities": [...],
    "required_skills": [...],
    "preferred_skills": [...],
    "salary_estimate": {...},
    "experience_levels": [...],
    "career_growth": [...],
    "interview_topics": [...],
    "industry_demand": "Very High",
    "growth_outlook": "Excellent..."
  },
  "related_guides": [...],
  "platform_urls": {
    "linkedin": "https://www.linkedin.com/jobs/search/?keywords=Python%20Developer",
    "indeed": "https://in.indeed.com/jobs?q=Python%20Developer",
    "naukri": "https://www.naukri.com/Python%20Developer-jobs",
    ...
  }
}
```

Used By: `templates/index.html` (search functionality)

POST /api/expand-roles

Purpose: Get related roles for a given job title

Request Body:

```
{
  "role": "Python Developer"
}
```

Response (Success - 200):

```
{
  "role": "Python Developer",
  "related_roles": [
    "Python Developer",
    "Backend Developer",
    "Django Developer",
    ...
  ]
}
```

5.1.3 Resume Analysis APIs

POST /api/analyze-resume

Purpose: Analyze resume and generate ATS score

Request Type: `multipart/form-data`

Form Data:

- `resume_text` (string): Resume text content
- `target_role` (string): Target job role (optional)
- `resume_file` (file): Upload file (.txt, .pdf, .docx)

Response (Success - 200):

```
{
  "ats_score": 78,
  "ranking": "Good",
  "ranking_description": "Your resume is solid but has room for improvement",
  "ats_breakdown": {
    "sections": 8.5,
    "contact_info": 4.5,
    "keyword_match": 22.3,
    "skills_match": 12.0,
    "experience": 16.5,
    "education": 4.0,
    "projects_achievements": 7.5,
    "grammar_language": 9.0,
    "length": 4.0,
    "ats_compatibility": 4.0
  },
  "sections_detected": {
    "contact": true,
    "summary": true,
    "experience": true,
    "education": true,
    "skills": true,
    "projects": false,
    "certifications": false,
    "achievements": true
  },
  "contact_info": {
    "email": "user@example.com",
    "phone": "+1234567890",
    "linkedin": "linkedin.com/in/johndoe",
    "github": "github.com/johndoe",
    "portfolio": null
  },
  "resume_skills": ["Python", "Django", "Flask", "SQL", "Docker", ...],
  "matched_keywords": ["python", "django", "sql", "git", ...],
  "missing_keywords": ["docker", "aws", "kubernetes", ...],
  "word_count": 650,
  "sentence_count": 42,
  "avg_words_per_sentence": 15.5,
  "action_verbs_found": 8,
  "action_verbs_list": ["developed", "implemented", "led", ...],
  "strengths": [
    "Strong technical skill set with 15 identified skills",
    "Excellent use of action verbs (8 found)",
    "Includes quantifiable achievements with metrics"
  ],
  "weaknesses": [
```

```

    "Missing projects section",
    "No GitHub profile URL"
  ],
  "missing_items": [
    "Projects section",
    "GitHub profile URL",
    "Certifications section"
  ],
  "suggestions": [
    {
      "section": "Projects",
      "current": "No projects section",
      "improvement": "Add 2-3 relevant projects...",
      "priority": "High",
      "impact": "+6% ATS score"
    }
  ],
  "skill_gap": {
    "target_role": "Python Developer",
    "required_skills": [...],
    "possessed": [...],
    "missing": [...],
    "match_percentage": 65.5
  },
  "readability": {
    "avg_words_per_sentence": 15.5,
    "total_words": 650,
    "total_sentences": 42,
    "reading_level": "Moderate"
  },
  "ats_issues": [],
  "grammar_issues": []
}

```

Scoring System (100 points total):

- Sections: 10 points
- Contact Info: 5 points
- Keyword Match: 30 points
- Skills Match: 15 points
- Experience: 20 points
- Education: 5 points
- Projects/Achievements: 10 points
- Grammar/Language: 10 points
- Length: 5 points
- ATS Compatibility: 5 points

Used By: <templates/index.html> (resume analyzer section)

POST /api/skill-gap

Purpose: Analyze skill gap for target role

Request Body:

```
{
  "role": "Python Developer",
  "skills": ["Python", "Django", "SQL"]
}
```

Response (Success - 200):

```
{
  "role": "Python Developer",
  "required_skills": [
    "Python", "Django", "Flask", "FastAPI", "REST API",
    "PostgreSQL", "MySQL", "Git", "Docker", "AWS", ...
  ],
  "possessed": ["Python", "Django", "SQL"],
  "missing": ["Flask", "FastAPI", "REST API", "PostgreSQL", ...],
  "match_percentage": 25.0,
  "critical_skills": ["Python", "SQL", "Git", "Docker", "AWS"],
  "important_skills": ["Kubernetes", "Terraform", "GraphQL", ...],
  "optional_skills": ["Redis", "Celery", "SQLAlchemy", ...],
  "learning_roadmap": [
    {
      "skill": "Flask",
      "type": "Framework",
      "difficulty": "Beginner-Intermediate",
      "time": "2-3 weeks",
      "resources": ["Flask Mega-Tutorial", "Flask Official Docs"]
    }
  ]
}
```

5.1.4 Interview Preparation APIs

POST /api/interview-prep

Purpose: Get role-specific interview questions

Request Body:

```
{
  "role": "Python Developer"
}
```

Response (Success - 200):

```
{
  "role": "Python Developer",
  "questions": {
    "technical": [
      {
        "q": "What are Python decorators and how do you use them?",
        "tips": "Explain function decorators, class decorators..."
      }
    ],
    "hr": [...],
    "coding": [...],
    "behavioral": [...]
  }
}
```

Question Categories:

- **Technical:** 8-10 role-specific technical questions
- **HR:** 4 general behavioral questions
- **Coding:** 3-5 programming challenges
- **Behavioral:** 2-3 situational questions

Used By: [templates/index.html](#) (interview prep section)

5.1.5 Career Insights APIs

POST /api/career-insights

Purpose: Get market data and career insights

Request Body:

```
{
  "role": "Data Scientist"
}
```

Response (Success - 200):

```
{
  "role": "Data Scientist",
  "insights": {
    "avg_salary_usd": 130000,
    "avg_salary_inr": "₹10,00,000 - ₹35,00,000",
    "demand_level": "Very High",
    "growth_rate": "+36% (2024-2030)",
    "hiring_trend": "Rapidly Increasing - AI/ML boom driving demand",
    "top_companies": [
      "Google", "Amazon", "Microsoft", "Meta", "Apple",
      "OpenAI", "Tesla", "JPMorgan"
    ],
    "future_outlook": "Exceptional - Every industry needs data-driven decision making...",
    "key_industries": [
      "Technology", "Finance", "Healthcare", "Retail",
      "Manufacturing", "Government"
    ],
    "entry_level_salary": "$80,000 - $110,000",
    "senior_level_salary": "$180,000 - $300,000+"
  }
}
```

5.1.6 Dashboard APIs

GET /api/dashboard

Purpose: Get user dashboard data

Headers: Requires authentication

Response (Success - 200):

```
{
  "saved_jobs": [...],
  "interview_calls": [...],
  "ats_scores": [...],
  "resume_versions": [...],
  "skill_progress": {...},
  "search_history": [...]
}
```

POST /api/dashboard/save-job

Purpose: Save a job to user's collection

Request Body:

```
{
  "guide": {
    "role_title": "Python Developer",
    "overview": "...",
    ...
  }
}
```

POST /api/dashboard/remove-job

Purpose: Remove saved job

Request Body:

```
{
  "role_title": "Python Developer"
}
```

POST /api/dashboard/update-skill-progress

Purpose: Update skill learning progress

Request Body:

```
{
  "skill": "Docker",
  "progress": 75
}
```

5.1.7 Feedback APIs

POST /api/feedback

Purpose: Submit user feedback

Request Body:

```
{
  "rating": 5,
  "comment": "Excellent platform!",
  "category": "general",
  "name": "John Doe",
  "email": "john@example.com",
  "is_anonymous": false
}
```

Response (Success - 201):

```
{
  "success": true,
  "message": "Thank you for your feedback!",
  "feedback_id": 123
}
```

Validation:

- Rating: 1-5 (required)
 - Comment: Non-empty string (required)
 - Category: general, feature, bug, improvement
-

GET /api/feedback

Purpose: Get all feedback (admin)

Query Parameters:

- `page` (int): Page number (default: 1)
- `per_page` (int): Items per page (default: 20)

Response (Success - 200):

```
{
  "success": true,
  "feedback": [...],
  "total": 150,
  "pages": 8,
  "current_page": 1,
  "average_rating": 4.3,
  "rating_distribution": {
    "5": 80,
    "4": 45,
    "3": 15,
    "2": 7,
    "1": 3
  }
}
```

GET /api/feedback/stats

Purpose: Get feedback statistics

Response (Success - 200):

```
{
  "success": true,
  "total_feedback": 150,
  "average_rating": 4.3,
  "recent_feedback_count": 25,
  "verified_reviews_count": 120,
  "rating_breakdown": {
    "excellent": 80,
    "good": 45,
    "average": 15,
    "poor": 10
  }
}
```

5.1.8 Resume Optimization API

POST /api/resume-optimize

Purpose: Generate optimized resume content

Request Body:

```
{
  "resume_text": "Full resume text here...",
  "target_role": "Python Developer"
}
```

Response (Success - 200):

```
{
  "optimized": {
    "summary": "Results-driven Python Developer with expertise in Python, Django, Flask...",
    "headline": "Python Developer | Python, Django, Flask | Building Scalable Solutions",
    "keywords_to_add": ["Docker", "AWS", "Kubernetes", ...],
    "section_improvements": [
      {
        "section": "Projects",
        "suggestion": "Add 2-3 projects...",
        "priority": "High"
      }
    ]
  },
  "original_score": 72,
  "suggestions": [...]
}
```

5.2 External APIs

5.2.1 RemoteOK API

Endpoint: <https://remoteok.com/api>

Method: GET

Authentication: None (public API)

Rate Limit: No official limit (respectful usage recommended)

Response Format:

```
[
  {
    "id": 12345,
    "epoch": 1689123456,
    "position": "Senior Python Developer",
    "company": "Tech Corp",
    "location": "Remote",
    "url": "https://remoteok.com/remote-jobs/...",
    "salary": "$120k - $180k",
    "description": "<p>Job description HTML</p>",
    "tags": ["python", "django", "postgresql"]
  }
]
```

Used By: `scrapers/remotekok_scraper.py`

5.2.2 Adzuna API

Endpoint: `https://api.adzuna.com/v1/api/jobs/{country}/search/1`

Method: GET

Authentication: App ID + API Key

Rate Limit: 1000 calls/month (free tier)

Parameters:

- `app_id` : Your application ID
- `app_key` : Your API key
- `what` : Job search query
- `where` : Location
- `results_per_page` : Number of results (max 20)

Used By: `utils/api_integrations.py`

5.2.3 JSearch API (RapidAPI)

Endpoint: `https://jsearch.p.rapidapi.com/search`

Method: GET

Authentication: X-RapidAPI-Key header

Rate Limit: Varies by plan

Headers:

- `X-RapidAPI-Key` : Your API key
- `X-RapidAPI-Host` : `jsearch.p.rapidapi.com`

Used By: `utils/api_integrations.py`

5.2.4 Google Gemini API

Endpoint: `https://generativelanguage.googleapis.com/v1beta/models/gemini-pro:generateContent`

Method: POST

Authentication: API Key

Rate Limit: 60 requests/minute (free tier)

Used By:

- N8N workflows (resume analysis, interview prep, career insights, feedback)
- `utils/api_integrations.py` (optional AI enhancement)

5.2.5 Hugging Face API

Endpoint: `https://api-inference.huggingface.co/models/{model_id}`

Method: POST

Authentication: Bearer token

Rate Limit: 1000 requests/month (free tier)

Used By: `utils/api_integrations.py` (resume classification)

5.2.6 ESCO API

Endpoint: `https://ec.europa.eu/esco/api/search`

Method: GET

Authentication: None

Rate Limit: No official limit

Used By: `utils/api_integrations.py` (skill lookup)

6. Environment Variables & Configuration

6.1 Complete Environment Variables List

Database Configuration

Variable	Purpose	Required	Default	Used In
<code>DATABASE_URL</code>	PostgreSQL connection string	No	<code>sqlite:///jobagent.db</code>	<code>app.py</code> , <code>docker-compose.yml</code>
<code>DB_PASSWORD</code>	Database password (Docker)	Yes	<code>secure_password_here</code>	<code>docker-compose.yml</code>

Security Configuration

Variable	Purpose	Required	Default	Used In
SECRET_KEY	Flask session encryption key	Yes	Random 24-char hex	app.py
JWT_SECRET_KEY	JWT token signing key	No	-	.env.example

AI/ML API Keys

Variable	Purpose	Required	Default	Used In
GEMINI_API_KEY	Google Gemini AI API	No	-	utils/api_integrations.py , N8N workflows
HUGGINGFACE_API_KEY	Hugging Face API	No	-	utils/api_integrations.py
OPENAI_API_KEY	OpenAI API (optional)	No	-	utils/api_integrations.py

Job Search APIs

Variable	Purpose	Required	Default	Used In
ADZUNA_API_KEY	Adzuna job search API	No	-	utils/api_integrations.py
ADZUNA_APP_ID	Adzuna application ID	No	-	utils/api_integrations.py
RAPIDAPI_KEY	RapidAPI key (JSearch)	No	-	utils/api_integrations.py

Email Services

Variable	Purpose	Required	Default	Used In
SMTP_HOST	SMTP server hostname	No	smtp.gmail.com	.env.example
SMTP_PORT	SMTP server port	No	587	.env.example
SMTP_USER	SMTP username/email	No	-	.env.example
SMTP_PASS	SMTP password/app password	No	-	.env.example

N8N Automation

Variable	Purpose	Required	Default	Used In
<code>N8N_WEBHOOK_URL</code>	N8N webhook endpoint	No	<code>http://localhost:5678/webhook</code>	<code>app.py</code> , <code>docker-compose.yml</code>
<code>N8N_API_KEY</code>	N8N API authentication	No	-	<code>.env.example</code>
<code>N8N_PASSWORD</code>	N8N admin password (Docker)	No	<code>n8n_admin_password</code>	<code>docker-compose.yml</code>

External APIs

Variable	Purpose	Required	Default	Used In
<code>CURRENCY_API_KEY</code>	Currency conversion API	No	-	<code>.env.example</code>
<code>IP_API_KEY</code>	IP geolocation API	No	-	<code>.env.example</code>

Application Settings

Variable	Purpose	Required	Default	Used In
<code>FLASK_ENV</code>	Flask environment	No	<code>development</code>	<code>app.py</code>
<code>FLASK_DEBUG</code>	Debug mode	No	<code>True</code>	<code>app.py</code>
<code>PORT</code>	Application port	No	<code>5000</code>	<code>app.py</code>

CORS Configuration

Variable	Purpose	Required	Default	Used In
<code>CORS_ORIGINS</code>	Allowed CORS origins	No	<code>http://localhost:3000,http://localhost:5000</code>	<code>.env.example</code>

Rate Limiting

Variable	Purpose	Required	Default	Used In
<code>RATE_LIMIT</code>	General rate limit	No	<code>100 per hour</code>	<code>.env.example</code>
<code>RATE_LIMIT_AUTH</code>	Auth endpoint rate limit	No	<code>10 per minute</code>	<code>.env.example</code>

Cache Configuration

Variable	Purpose	Required	Default	Used In
CACHE_TTL	Cache time-to-live (seconds)	No	3600	.env.example
REDIS_URL	Redis connection string	No	redis://localhost:6379/0	.env.example
REDIS_PASSWORD	Redis password (Docker)	No	redis_password_here	docker-compose.yml

Monitoring

Variable	Purpose	Required	Default	Used In
SENTRY_DSN	Sentry error tracking	No	-	.env.example
LOG_LEVEL	Logging verbosity	No	INFO	config.py

Scraping Configuration

Variable	Purpose	Required	Default	Used In
FIRECRAWL_API_KEY	Firecrawl scraping API	No	-	config.py , scrapers/wellfound_scraper.py
DEFAULT_LIMIT	Default job scrape limit	No	50	config.py
REQUEST_TIMEOUT	HTTP request timeout (seconds)	No	10	config.py
RETRY_ATTEMPTS	Number of retry attempts	No	3	config.py
RETRY_DELAY	Delay between retries (seconds)	No	2	config.py
RATE_LIMIT_DELAY	Delay between requests (seconds)	No	1.0	config.py

6.2 Configuration Loading Flow

```
# 1. Load .env file
load_dotenv() # From python-dotenv

# 2. Config dataclass loads defaults
config = Config()

# 3. __post_init__ overrides with env vars
config.firecrawl_api_key = os.getenv('FIRECRAWL_API_KEY')
config.default_limit = int(os.getenv('DEFAULT_LIMIT', 50))

# 4. Flask app config
app.config['SQLALCHEMY_DATABASE_URI'] = os.environ.get('DATABASE_URL')
app.secret_key = os.environ.get('SECRET_KEY', os.urandom(24).hex())
```

7. Database Schema

7.1 Entity Relationship Diagram

users		
id (PK)	Integer	
email (UNIQUE, IDX)	String(255)	
username (UNIQUE)	String(100)	
password_hash	String(512)	
is_active	Boolean	
is_verified	Boolean	
created_at	DateTime	
updated_at	DateTime	
last_login	DateTime	
login_attempts	Integer	
locked_until	DateTime	

|
| 1:1
▼

user_profiles		
id (PK)	Integer	
user_id (FK, IDX)	Integer → users.id	
full_name	String(200)	
phone	String(50)	
location	String(200)	
headline	String(300)	
bio	Text	
linkedin_url	String(500)	
github_url	String(500)	
portfolio_url	String(500)	
avatar_url	String(500)	
resume_file_path	String(500)	
resume_text	Text	
theme_preference	String(20)	
preferred_roles	Text (JSON array)	
skills	Text (JSON array)	
experience_level	String(50)	
created_at	DateTime	
updated_at	DateTime	

| 1:1



user_preferences	
id (PK)	Integer
user_id (FK, IDX)	Integer → users.id
email_notifications	Boolean
job_alerts	Boolean
weekly_digest	Boolean
theme	String(20)
language	String(10)
search_radius_km	Integer
preferred_locations	Text (JSON array)
excluded_keywords	Text (JSON array)
created_at	DateTime
updated_at	DateTime

saved_jobs	
id (PK)	Integer
user_id (FK, IDX)	Integer → users.id
role_title	String(300)
guide_data	Text (JSON)
company	String(200)
job_url	String(1000)
notes	Text
status	String(50)
saved_at	DateTime
updated_at	DateTime
Index: idx_user_role (user_id, role_title)	

job_applications	
id (PK)	Integer
user_id (FK, IDX)	Integer → users.id
saved_job_id (FK)	Integer → saved_jobs.id
company	String(200)
role	String(300)
location	String(200)
job_url	String(1000)
salary_range	String(100)

status	String(50)	
applied_date	DateTime	
last_updated	DateTime	
notes	Text	
resume_used	String(500)	
Index: idx_app_user_status (user_id, status)		

	interview_history	
id (PK)	Integer	
user_id (FK, IDX)	Integer → users.id	
application_id (FK)	Integer → job_applications.id	
company	String(200)	
role	String(300)	
interview_type	String(100)	
interview_date	DateTime	
duration_minutes	Integer	
questions_asked	Text (JSON array)	
feedback	Text	
rating	Integer (1-5)	
status	String(50)	
notes	Text	
created_at	DateTime	
Index: idx_interview_user_date (user_id, interview_date)		

	user_sessions	
id (PK)	Integer	
user_id (FK, IDX)	Integer → users.id	
session_token (IDX)	String(256)	
ip_address	String(50)	
user_agent	String(500)	
is_active	Boolean	
created_at	DateTime	
expires_at	DateTime	
last_activity	DateTime	

	audit_logs	

id (PK)	Integer
user_id (FK, IDX)	Integer → users.id
action (IDX)	String(100)
resource_type	String(100)
resource_id	Integer
details	Text (JSON)
ip_address	String(50)
user_agent	String(500)
created_at	DateTime
Index: idx_audit_user_action (user_id, action)	

password_reset_tokens	
id (PK)	Integer
user_id (FK, IDX)	Integer → users.id
token (UNIQUE, IDX)	String(256)
is_used	Boolean
expires_at	DateTime
created_at	DateTime

feedback	
id (PK)	Integer
user_id (FK, IDX)	Integer → users.id
name	String(200)
email	String(255)
rating	Integer (1-5)
comment	Text
category	String(100)
is_anonymous	Boolean
is_verified	Boolean
is_helpful	Integer
is_not_helpful	Integer
status	String(50)
admin_response	Text
responded_at	DateTime
created_at (IDX)	DateTime
updated_at	DateTime
Indexes: idx_feedback_rating, idx_feedback_created, idx_feedback_status	

career_insight_cache	
id (PK)	Integer
role_key (UNIQUE, IDX)	String(200)
data	Text (JSON)
source	String(100)
expires_at (IDX)	DateTime
created_at	DateTime
updated_at	DateTime

notifications	
id (PK)	Integer
user_id (FK, IDX)	Integer → users.id
type	String(50)
title	String(300)
message	Text
data	Text (JSON)
is_read (IDX)	Boolean
is_sent	Boolean
sent_at	DateTime
created_at (IDX)	DateTime
Index: idx_notification_user_read (user_id, is_read)	

analytics	
id (PK)	Integer
user_id (FK, IDX)	Integer → users.id
event_type (IDX)	String(100)
event_data	Text (JSON)
ip_address	String(50)
user_agent	String(500)
session_id (IDX)	String(100)
created_at (IDX)	DateTime
Index: idx_analytics_event_type (event_type, created_at)	

7.2 Table Descriptions

users

Purpose: Store user account information

Row Count: Variable (user base)

Indexes: email, username

Relationships: 1:1 with user_profiles, 1:N with saved_jobs, job_applications, interview_history, user_sessions, audit_logs, feedback

user_profiles

Purpose: Extended user profile information

Key Fields: resume_text, skills (JSON), preferred_roles (JSON)

Relationships: Belongs to user

saved_jobs

Purpose: User-saved job roles and guides

Status Values: saved, applied, interviewing, offer, rejected

Relationships: Belongs to user

job_applications

Purpose: Track job applications

Status Values: applied, screening, interview, offer, rejected, accepted

Relationships: Belongs to user, optionally linked to saved_jobs

interview_history

Purpose: Record interview sessions and outcomes

Interview Types: technical, hr, coding, behavioral, final

Status Values: scheduled, completed, cancelled, rescheduled

Relationships: Belongs to user, optionally linked to job_applications

user_sessions

Purpose: Track active user sessions

Token Generation: `secrets.token_urlsafe(64)`

Expiration: 1 day (default) or 30 days (remember me)

Relationships: Belongs to user

audit_logs

Purpose: Security and compliance tracking

Actions: account_created, login, logout, password_reset_requested, password_reset_completed, job_saved, etc.

Relationships: Belongs to user (nullable for anonymous actions)

feedback

Purpose: Store user feedback and ratings
Rating: 1-5 scale
Status Values: new, reviewed, in_progress, resolved, closed
Relationships: Belongs to user (nullable for anonymous feedback)

career_insight_cache

Purpose: Cache AI-generated career insights
TTL: 24 hours (default)
Purpose: Reduce API calls and improve performance

notifications

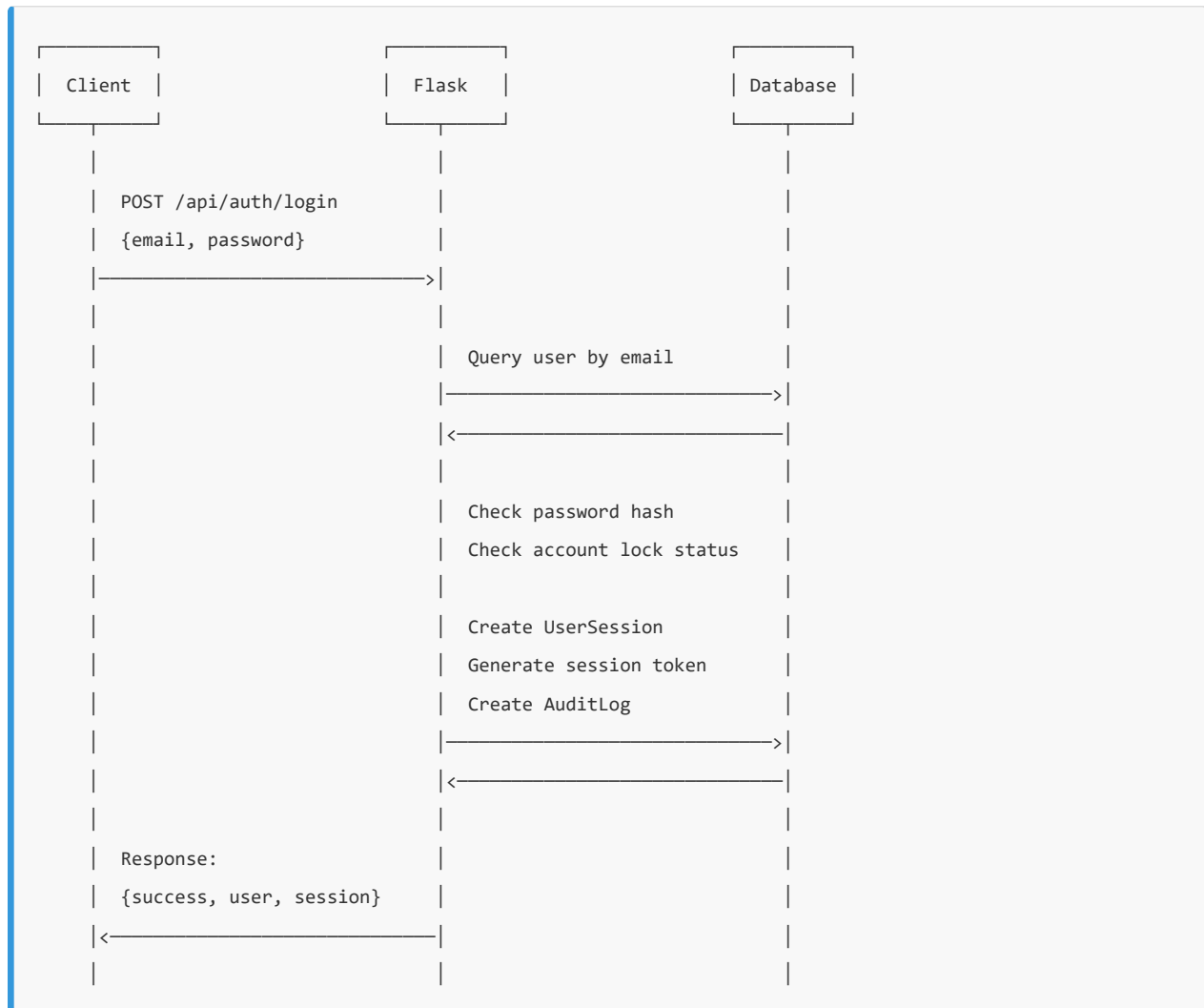
Purpose: In-app notifications for users
Types: job_alert, interview_reminder, feedback_response, system
Relationships: Belongs to user

analytics

Purpose: Track user behavior and events
Event Types: page_view, feature_used, search_performed, resume_analyzed, etc.
Relationships: Belongs to user (nullable)

8. Authentication & Authorization

8.1 Authentication Flow



8.2 Security Features

Password Security

- **Hashing:** Werkzeug `generate_password_hash` (pbkdf2)
- **Requirements:**
 - 8-128 characters
 - At least one uppercase letter
 - At least one lowercase letter
 - At least one number
 - At least one special character
 - No common patterns (password, 123456, qwerty, etc.)

Account Lockout

- **Trigger:** 5 failed login attempts
- **Duration:** 15 minutes
- **Auto-unlock:** Yes (after 15 minutes)
- **Reset:** On successful login

Session Management

- **Token Generation:** `secrets.token_urlsafe(64)` (cryptographically secure)
- **Session Duration:**
 - Default: 1 day
 - Remember me: 30 days
- **Storage:** Database (user_sessions table)
- **Invalidation:** On logout or password reset

Input Sanitization

```
def sanitize_input(text: str) -> str:
    """Prevent XSS attacks"""
    # Strip HTML tags
    text = re.sub(r'<[^>]*>', '', text)
    # Remove script tags
    text = re.sub(r'<script[^>]*>.*?</script>', '', text, flags=re.DOTALL)
    # Remove event handlers
    text = re.sub(r'\bon\bw+\s*=\s*["\'](?:\"|\'|\\\\)*["\']', '', text, flags=re.IGNORECASE)
    return text.strip()
```

Audit Logging

All security events are logged:

- Account creation
- Login attempts (success/failure)
- Logout
- Password reset requests
- Password reset completion
- Job saves/removals
- Profile updates

Logged Data:

- User ID
- Action type
- Resource type and ID
- IP address
- User agent
- Timestamp

8.3 Protected Routes

```
# Authentication required
@app.route('/api/auth/me')
@login_required
def api_auth_me():
    return jsonify(get_user_dashboard_data(current_user))

@app.route('/api/auth/logout', methods=['POST'])
@login_required
def api_auth_logout():
    logout_user_session(current_user)
    logout_user()
    return jsonify({'success': True})
```

8.4 Authorization Levels

Level	Access	Implementation
Public	Home page, login, signup	No authentication required
User	Dashboard, resume analysis, job search	@login_required decorator
Admin	Feedback management, user management	Not implemented (future)

9. Frontend Architecture

9.1 Page Structure

index.html (Main Application)

Purpose: Single-page application with multiple sections

Sections:

- 1. **Navigation:** Logo, menu items, user profile
- 2. **Hero Section:** Search bar for job roles
- 3. **Role Guide:** Detailed role information
- 4. **Resume Analyzer:** File upload and text analysis
- 5. **Skill Gap Analysis:** Missing skills and learning roadmap
- 6. **Interview Preparation:** Role-specific questions
- 7. **Career Insights:** Market data and trends
- 8. **Dashboard:** User's saved jobs, applications, interviews
- 9. **Feedback:** Rating and comment form

Technologies: HTML5, CSS3, JavaScript (ES6+), Jinja2 templating

login.html

Purpose: User login page

Features:

- Email/password form
- "Remember me" checkbox
- Link to signup and forgot password
- Client-side validation

signup.html

Purpose: User registration page

Features:

- Email, username, password, confirm password fields
- Real-time password strength indicator
- Client-side validation
- Link to login

forgot_password.html

Purpose: Password reset request

Features:

- Email input
- Success message (doesn't reveal if email exists)

reset_password.html

Purpose: Password reset form

Features:

- Token validation (from URL parameter)
- New password and confirmation fields
- Password strength requirements

9.2 Frontend JavaScript (app.js)

Key Functions:

```

// API Communication
async function apiCall(endpoint, method = 'GET', data = null) {
  const options = {
    method,
    headers: {
      'Content-Type': 'application/json',
    },
    credentials: 'include' // Include cookies for session
  };

  if (data) {
    options.body = JSON.stringify(data);
  }

  const response = await fetch(endpoint, options);
  return await response.json();
}

// Resume Analysis
async function analyzeResume() {
  const formData = new FormData();
  formData.append('resume_text', resumeText);
  formData.append('target_role', targetRole);
  formData.append('resume_file', fileInput.files[0]);

  const response = await fetch('/api/analyze-resume', {
    method: 'POST',
    body: formData
  });

  return await response.json();
}

// Search Role
async function searchRole(query) {
  return await apiCall('/api/search', 'POST', {
    query: query,
    roles: []
  });
}

```

9.3 State Management

Client-Side State:

```
// User session
let currentUser = null;

// Search state
let currentQuery = '';
let currentGuide = null;

// Resume analysis
let resumeAnalysisResult = null;

// Dashboard data
let savedJobs = [];
let applications = [];
let interviews = [];
```

Server-Side State:

```
# In-memory session store (anonymous users)
user_sessions: Dict[str, Dict[str, Any]] = {
    'saved_jobs': [],
    'interview_calls': [],
    'ats_scores': [],
    'resume_versions': [],
    'skill_progress': {},
    'search_history': []
}
```

9.4 CSS Architecture

File: `static/css/style.css`

Organization:

- CSS Variables (colors, fonts, spacing)
- Reset/Normalize
- Layout (grid, flexbox)
- Components (buttons, cards, forms)
- Sections (hero, dashboard, analyzer)
- Utilities (spacing, typography)
- Animations
- Responsive breakpoints

Theme Support: Dark/Light mode via CSS custom properties

10. Backend Architecture

10.1 Flask Application Structure

```
# app.py organization
app = Flask(__name__)

# 1. Configuration
app.config['SQLALCHEMY_DATABASE_URI'] = ...
app.config['MAX_CONTENT_LENGTH'] = 16 * 1024 * 1024 # 16MB

# 2. Extensions
db.init_app(app)
login_manager.init_app(app)

# 3. In-Memory Stores
user_sessions: Dict[str, Dict[str, Any]] = {}

# 4. Data Constants
ROLE_MAP: Dict[str, List[str]] = {...}
SKILLS_DATABASE: Dict[str, List[str]] = {...}
ROLE_GUIDES_DB: Dict[str, Dict[str, Any]] = {...}
INTERVIEW_QUESTIONS: Dict[str, Dict[str, List]] = {...}
CAREER_INSIGHTS_DB: Dict[str, Dict[str, Any]] = {...}
LEARNING_RESOURCES: Dict[str, Dict[str, str]] = {...}

# 5. Helper Functions
def get_session_id() -> str: ...
def get_user_data() -> Dict[str, Any]: ...
def get_related_roles(role: str) -> List[str]: ...
def get_role_guide(role: str) -> Dict[str, Any]: ...
def analyze_resume_text(text: str, target_role: str) -> Dict[str, Any]: ...
def get_interview_questions(role: str) -> Dict[str, Any]: ...
def get_career_insights(role: str) -> Dict[str, Any]: ...

# 6. Routes
@app.route('/')
def index(): ...

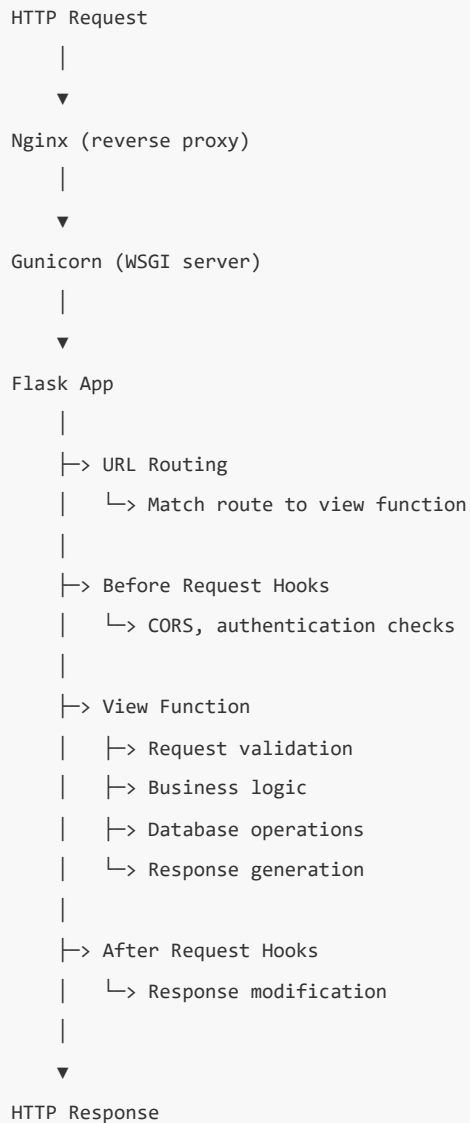
@app.route('/api/auth/signup', methods=['POST'])
def api_auth_signup(): ...

@app.route('/api/search', methods=['POST'])
def api_search(): ...

# ... more routes
```

```
# 7. Application Entry Point
if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=5000)
```

10.2 Request Processing Flow



10.3 Business Logic Modules

Resume Analysis Engine

Location: `app.py` lines 907-1465

Function: `analyze_resume_text(text: str, target_role: str)`

10 Components (100 points total):

1. **Sections** (10%): Contact, summary, experience, education, skills, projects, certifications, achievements
2. **Contact Info** (5%): Email, phone, LinkedIn, GitHub, portfolio

3. **Keyword Match** (30%): Role-specific keyword matching
4. **Skills Match** (15%): Technical skills identification
5. **Experience** (20%): Job titles, company names, experience indicators
6. **Education** (5%): Degree level, certifications
7. **Projects/Achievements** (10%): Project section, quantifiable achievements
8. **Grammar/Language** (10%): Repeated words, long sentences, professional language
9. **Length** (5%): Optimal word count (400-800 words)
10. **ATS Compatibility** (5%): Standard headers, plain text format

Output:

- ATS score (0-100)
- Ranking (Excellent/Good/Average/Below Average/Poor)
- Detailed breakdown
- Strengths and weaknesses
- Actionable suggestions
- Skill gap analysis

Role Guide Engine

Location: `app.py` lines 386-868

Function: `get_role_guide(role: str)`

13 Pre-defined Roles:

1. Frontend Developer
2. Python Developer
3. Data Scientist
4. Full Stack Developer
5. DevOps Engineer
6. Machine Learning Engineer
7. Data Analyst
8. UI/UX Designer
9. AI Engineer
10. Mobile Developer
11. Blockchain Developer
12. Cloud Architect
13. Product Manager

Guide Contents:

- Overview
- Responsibilities (8 items)
- Required skills
- Preferred skills
- Salary estimates (USD, INR, EUR)
- Experience levels
- Career growth path
- Interview topics
- Industry demand
- Growth outlook

Fallback: Generic template for unknown roles

Interview Question Engine

Location: `app.py` lines 1472-1757

Function: `get_interview_questions(role: str)`

Question Categories:

1. **Technical:** 8-10 role-specific questions
2. **HR:** 4 general behavioral questions
3. **Coding:** 3-5 programming challenges
4. **Behavioral:** 2-3 situational questions

Question Structure:

```
{
  "q": "Question text",
  "tips": "What the interviewer is looking for..."
}
```

500+ Total Questions across 8 role categories

11. N8N Workflows

11.1 Workflow Overview

The project uses 5 N8N workflows for AI-powered automation:

1. **Resume Analysis Workflow** - AI-powered resume scoring
2. **Job Recommendations Workflow** - Daily personalized job matching
3. **Interview Preparation Workflow** - AI-generated interview questions
4. **Career Insights Workflow** - Market data and trends
5. **Feedback Processing Workflow** - Sentiment analysis and routing

11.2 Resume Analysis Workflow

File: `n8n/workflows/resume_analysis_workflow.json`

Trigger: Webhook (POST `/resume-analysis`)

Nodes: 5

Node Flow

Webhook → AI Resume Analysis → Process Analysis → Save to Database → Respond

Node Details

1. Webhook (`n8n-nodes-base.webhook`)

- Path: `resume-analysis`
- Method: POST only
- Input: `{target_role, resume_text}`

2. AI Resume Analysis (`n8n-nodes-base.googleAI`)

- Model: `gemini-pro`
- Temperature: 0.3 (low for consistency)
- System Prompt: Expert ATS analyzer
- User Prompt: Analyze resume for target role, provide JSON with 13 fields

3. Process Analysis (`n8n-nodes-base.code`)

- Language: JavaScript
- Function: Parse AI response, extract JSON, add metadata
- Error Handling: Fallback structure if parsing fails

4. Save to Database (`n8n-nodes-base.postgres`)

- Operation: INSERT
- Table: `resume_analyses`
- Columns: user_id, role, ats_score, analysis_data, source, created_at

5. Respond to Webhook (`n8n-nodes-base.respondToWebhook`)

- Response Code: 200
- Body: Complete analysis JSON

Integration Points

- **Called By:** Flask app (future integration)
 - **Database:** PostgreSQL
 - **AI Service:** Google Gemini API
-

11.3 Job Recommendations Workflow

File: `n8n/workflows/job_recommendations_workflow.json`

Trigger: Schedule (Daily at 9:00 AM)

Nodes: 5

Node Flow

Daily Schedule → Get Active Users → AI Job Matching → Structure Recommendations → Save Recommendations → Send Email

Node Details

1. Daily Schedule (`n8n-nodes-base.scheduleTrigger`)
 - Cron: `0 9 * * *` (9:00 AM daily)
 - Timezone: Server timezone
2. Get Active Users (`n8n-nodes-base.postgres`)
 - Operation: SELECT
 - Table: `user_profiles`
 - Filter: Updated in last 7 days
 - Columns: user_id, skills, preferred_roles, experience_level, location
3. AI Job Matching (`n8n-nodes-base.googleAI`)
 - Model: `gemini-pro`
 - Temperature: 0.5 (moderate creativity)
 - Prompt: Generate 5-10 job recommendations based on user profile
4. Structure Recommendations (`n8n-nodes-base.code`)
 - Function: Parse AI response, add user context and timestamp
5. Save Recommendations (`n8n-nodes-base.postgres`)
 - Operation: INSERT
 - Table: `job_recommendations`
 - Columns: user_id, recommendations_data, created_at
6. Send Email Notification (`n8n-nodes-base.emailSend`)
 - To: User email
 - Subject: "Your Daily Job Recommendations - JobAgent"
 - Body: Personalized email with job count and login link

Integration Points

- Database: PostgreSQL (user_profiles, job_recommendations)
- AI Service: Google Gemini API
- Email: SMTP (configured in N8N)

11.4 Interview Preparation Workflow

File: `n8n/workflows/interview_prep_workflow.json`

Trigger: Webhook (POST `/interview-prep`)

Nodes: 5

Node Flow

Webhook → AI Interview Questions → Structure Questions → Save to Database → Respond

Node Details

1. Webhook (`n8n-nodes-base.webhook`)
 - Path: `interview-prep`
 - Method: POST only
 - Input: `{role, company, experience_level, industry}`
2. AI Interview Questions (`n8n-nodes-base.googleAI`)
 - Model: `gemini-pro`
 - Temperature: 0.7 (higher creativity)
 - Prompt: Generate questions for 4 categories with detailed answers
3. Structure Questions (`n8n-nodes-base.code`)
 - Function: Parse AI response, add metadata
4. Save to Database (`n8n-nodes-base.postgres`)
 - Operation: INSERT
 - Table: `interview_sessions`
 - Columns: user_id, role, company, questions_data, created_at
5. Respond to Webhook (`n8n-nodes-base.respondToWebhook`)
 - Response: Structured interview questions JSON

Question Format

```
{
  "technical": [
    {
      "question": "...",
      "what_interviewer_looks_for": "...",
      "key_points_to_cover": [...],
      "sample_strong_answer": "...",
      "common_mistakes": [...],
      "difficulty": "Medium",
      "estimated_time": 5
    }
  ]
}
```

11.5 Career Insights Workflow

File: `n8n/workflows/career_insights_workflow.json`
Trigger: Webhook (POST `/career-insights`)
Nodes: 5

Node Flow

Webhook → AI Career Insights → Structure Insights → Save to Database → Respond

Node Details

1. Webhook (`n8n-nodes-base.webhook`)

- Path: `career-insights`
- Method: POST only
- Input: `{role, country}`

2. AI Career Insights (`n8n-nodes-base.googleAI`)

- Model: `gemini-pro`
- Temperature: 0.3 (factual, low creativity)
- Prompt: Generate market data, salary ranges, top companies, growth trends

3. Structure Insights (`n8n-nodes-base.code`)

- Function: Parse AI response, add metadata

4. Save to Database (`n8n-nodes-base.postgres`)

- Operation: INSERT
- Table: `career_insights`
- Columns: role, insights_data, country, source, created_at

5. Respond to Webhook (`n8n-nodes-base.respondToWebhook`)

- Response: Career insights JSON

Insights Format

```
{
  "average_salary_usd": 130000,
  "average_salary_inr": "₹10,00,000 - ₹35,00,000",
  "entry_level_salary": "$80,000 - $110,000",
  "senior_level_salary": "$180,000 - $300,000+",
  "demand_level": "Very High",
  "growth_rate": "+36% (2024-2030)",
  "hiring_trend": "Rapidly Increasing...",
  "top_companies": ["Google", "Amazon", ...],
  "future_outlook": "...",
  "key_industries": [...],
  "required_skills": [...],
  "certifications": [...],
  "career_path": [...]
}
```

11.6 Feedback Processing Workflow

File: `n8n/workflows/feedback_processing_workflow.json`

Trigger: Webhook (POST `/feedback`)

Nodes: 7

Node Flow

Webhook → AI Feedback Analysis → Process Feedback → Save Feedback → Critical Priority? → [Send Alert Email] OR [Send Acknowledgment] → Respond

Node Details

1. Webhook (`n8n-nodes-base.webhook`)

- Path: `feedback`
- Method: POST only
- Input: `{rating, comment, category, name, email}`

2. AI Feedback Analysis (`n8n-nodes-base.googleAI`)

- Model: `gemini-pro`
- Temperature: 0.5
- Prompt: Analyze sentiment, priority, category, key points

3. Process Feedback (`n8n-nodes-base.code`)

- Function: Combine AI analysis with original data
- Status: Set to 'urgent' if priority is critical

4. Save Feedback (`n8n-nodes-base.postgres`)

- Operation: INSERT
- Table: `feedback`
- Columns: user_id, name, email, rating, comment, category, sentiment, priority, analysis_data, status, created_at

5. Critical Priority? (`n8n-nodes-base.if`)

- Condition: `priority == 'critical'`
- True Branch: Send Alert Email
- False Branch: Send Acknowledgment

6. Send Alert Email (`n8n-nodes-base.emailSend`)

- To: `admin@jobagent.com`
- Subject: "🚨 Critical Feedback Alert - JobAgent"
- Trigger: Only for critical priority feedback

7. Send Acknowledgment (`n8n-nodes-base.emailSend`)

- To: User email
- Subject: "Thank you for your feedback - JobAgent"
- Body: Includes AI-generated suggested response

8. Respond to Webhook (`n8n-nodes-base.respondToWebhook`)

- Response: Success confirmation

Analysis Output

```
{
  "sentiment": "positive",
  "priority": "medium",
  "category_suggested": "feature",
  "key_points": ["Great platform", "Would like more features"],
  "action_required": true,
  "suggested_response": "Thank you for your feedback...",
  "tags": ["feature-request", "positive"]
}
```

12. Dependencies

12.1 Core Dependencies

Package	Version	Purpose	Usage
flask	3.0.0	Web framework	Main application server
flask-login	0.6.3	Session management	User authentication
flask-sqlalchemy	3.1.1	ORM	Database operations
flask-cors	4.0.0	CORS handling	Cross-origin requests
python-dotenv	1.0.0	Environment variables	Configuration loading

12.2 Database Dependencies

Package	Version	Purpose	Usage
sqlalchemy	2.0.23	Database toolkit	ORM and query builder

12.3 HTTP & API Dependencies

Package	Version	Purpose	Usage
requests	2.31.0	HTTP client	Synchronous API calls
httpx	0.25.2	HTTP client	Async API calls
aiohttp	3.9.1	HTTP framework	Async web scraping

12.4 Data Processing Dependencies

Package	Version	Purpose	Usage
pandas	2.2.2	Data manipulation	Job data processing, Excel export
numpy	<2.0.0	Numerical computing	Data analysis

12.5 Web Scraping Dependencies

Package	Version	Purpose	Usage
beautifulsoup4	4.12.2	HTML parsing	Naukri scraping
lxml	4.9.3	XML/HTML parser	BeautifulSoup backend
firecrawl-py	0.0.16	Web scraping	Wellfound scraping
selenium	-	Browser automation	Dynamic content scraping
playwright	-	Browser automation	Wellfound API interception

12.6 AI/ML Dependencies

Package	Version	Purpose	Usage
google-generativeai	0.3.2	Gemini AI API	Resume analysis, interview questions, career insights
openai	1.6.1	OpenAI API	Optional enhanced features
huggingface-hub	0.19.4	Hugging Face API	Resume classification

12.7 Document Processing Dependencies

Package	Version	Purpose	Usage
python-docx	1.1.0	DOCX processing	Resume file parsing
PyPDF2	3.0.1	PDF processing	Resume file parsing
pdfplumber	0.10.3	PDF extraction	Advanced PDF text extraction

12.8 Utility Dependencies

Package	Version	Purpose	Usage
python-dateutil	2.8.2	Date parsing	Date handling
pydantic	2.5.3	Data validation	Schema validation
pydantic-settings	2.1.0	Settings management	Configuration
tenacity	8.2.3	Retry logic	API retry mechanism
cachetools	5.3.2	Caching	API response caching

12.9 Security Dependencies

Package	Version	Purpose	Usage
python-jose[cryptography]	3.3.0	JWT handling	Token generation/validation
passlib[bcrypt]	1.7.4	Password hashing	Secure password storage
email-validator	2.1.0	Email validation	Email format validation

12.10 Testing Dependencies

Package	Version	Purpose	Usage
pytest	7.4.0	Testing framework	Unit and integration tests
pytest-cov	4.1.0	Coverage reporting	Test coverage analysis
pytest-asyncio	0.21.1	Async testing	Async test support

13. Project Resource Mapping

13.1 Static Assets

CSS Files

File	Purpose	Size	Used In
static/css/style.css	Main stylesheet	~15KB	All pages

Key Features:

- CSS custom properties for theming
- Responsive grid layouts
- Component styles (buttons, cards, forms)
- Animation keyframes
- Dark/light mode support

JavaScript Files

File	Purpose	Size	Used In
static/js/app.js	Main application logic	~25KB	All pages

Key Functions:

- API communication
- Form handling
- DOM manipulation
- Event listeners
- State management

13.2 HTML Templates

Template	Purpose	Routes
templates/index.html	Main SPA	/
templates/login.html	Login page	/login
templates/signup.html	Registration page	/signup
templates/forgot_password.html	Password reset request	/forgot-password
templates/reset_password.html	Password reset form	/reset-password/<token>

13.3 N8N Workflows

Workflow File	Purpose	Trigger	Nodes
resume_analysis_workflow.json	AI resume analysis	Webhook	5
job_recommendations_workflow.json	Daily job matching	Schedule (daily 9AM)	6
interview_prep_workflow.json	Interview question generation	Webhook	5
career_insights_workflow.json	Career market insights	Webhook	5
feedback_processing_workflow.json	Feedback analysis	Webhook	8

13.4 Configuration Files

File	Purpose	Format
<code>.env.example</code>	Environment variable template	Key-value pairs
<code>config.py</code>	Application configuration	Python dataclass
<code>Dockerfile</code>	Container image definition	Dockerfile
<code>docker-compose.yml</code>	Multi-container orchestration	YAML
<code>requirements.txt</code>	Python dependencies	pip format

13.5 External Services

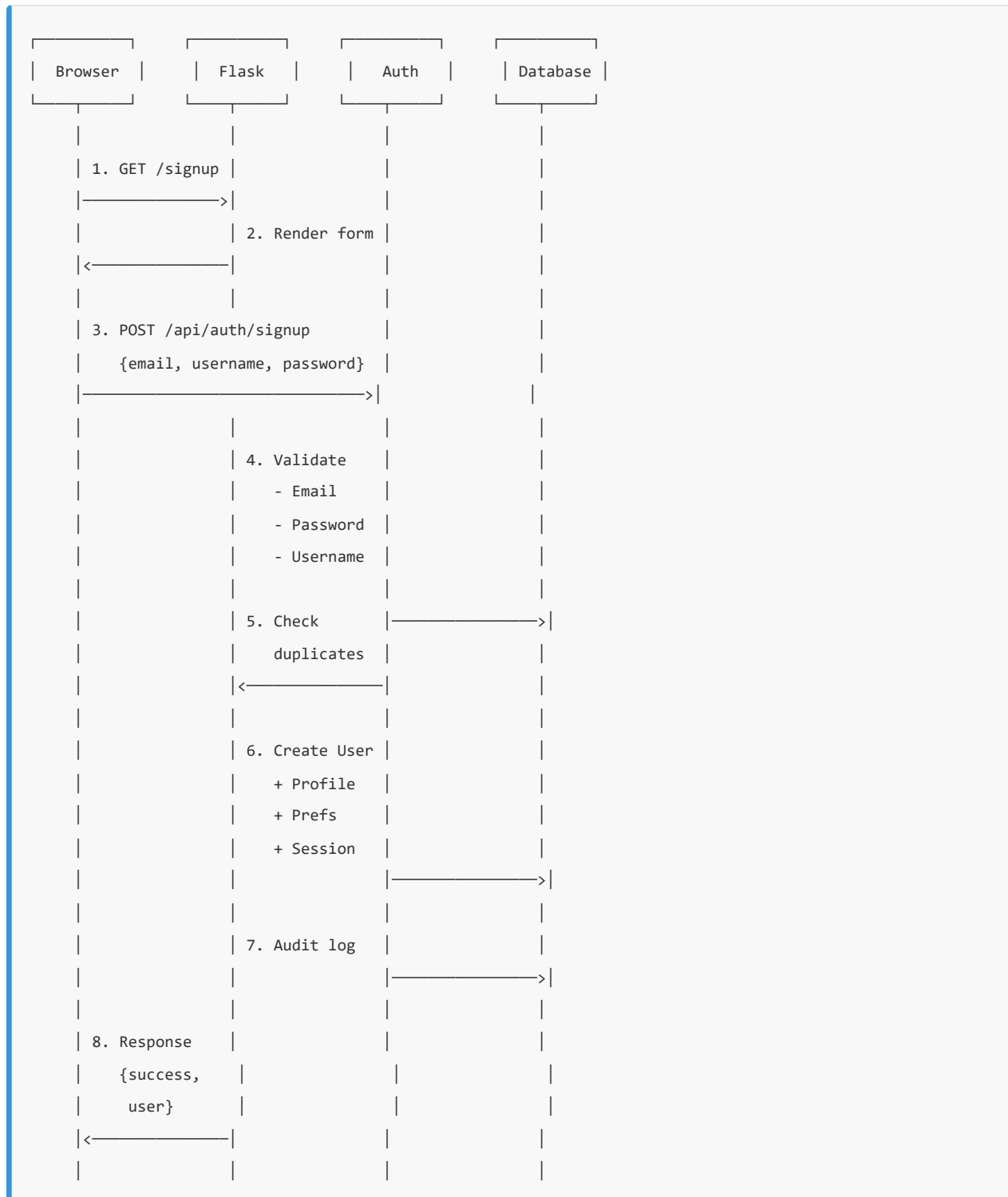
Service	Purpose	API Key Required	Free Tier
Google Gemini	AI analysis and generation	Yes	60 req/min
Hugging Face	Resume classification	Yes	1000 req/month
OpenAI	Enhanced AI features	Yes	Pay-as-you-go
Adzuna	Job search API	Yes	1000 req/month
RapidAPI (JSearch)	Job search API	Yes	Varies
RemoteOK	Job listings	No	Unlimited
Naukri	Job scraping	No	Unlimited (scraping)
Wellfound	Job scraping	No	Unlimited (scraping)
ESCO	Skills database	No	Unlimited

13.6 Third-Party Libraries

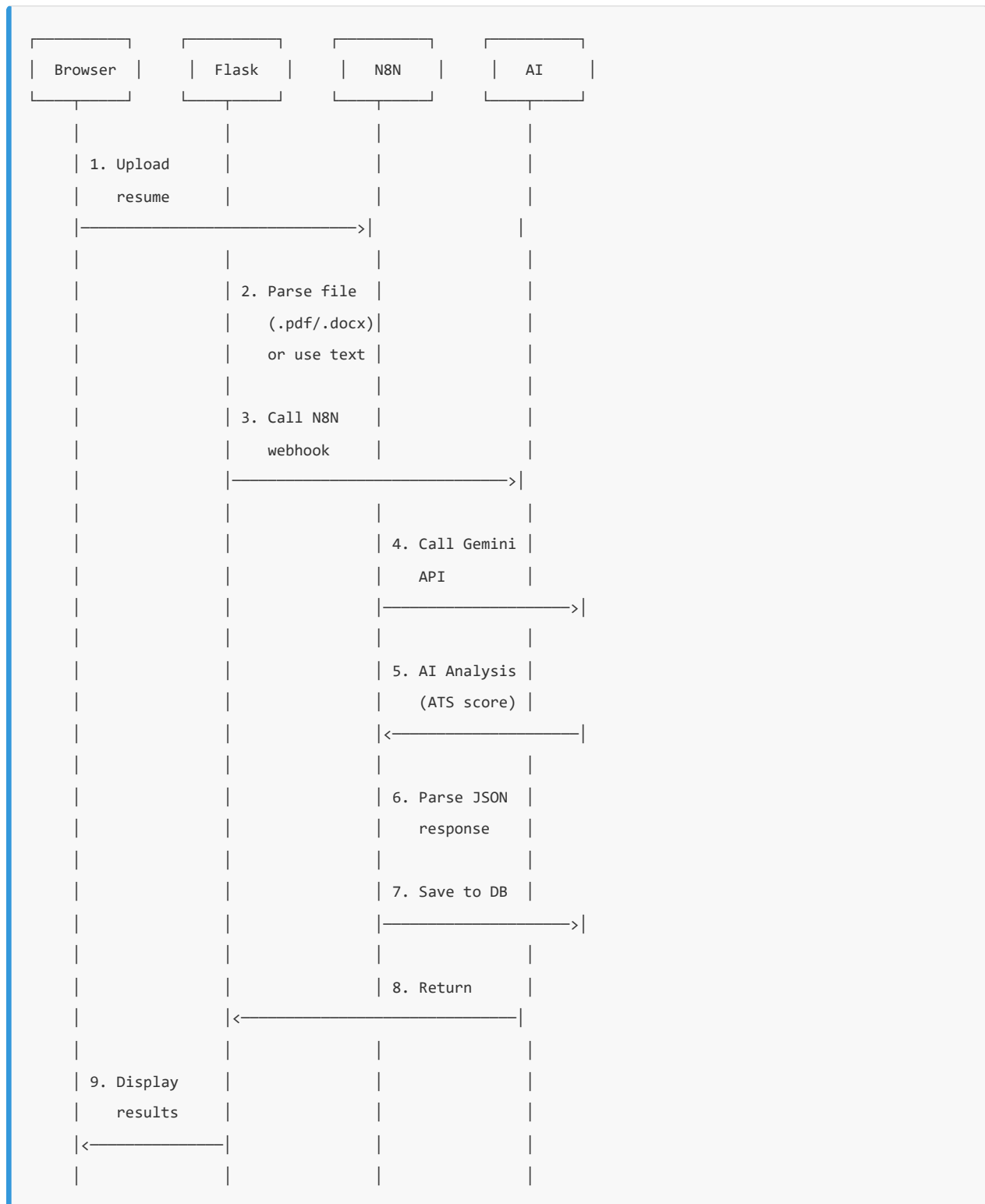
Library	Purpose	Integration Point
BeautifulSoup4	HTML parsing	<code>scrapers/base_scraper.py</code>
Selenium	Browser automation	<code>scrapers/base_scraper.py</code>
Playwright	Browser automation	<code>scrapers/wellfound_scraper.py</code>
Flask-Login	Authentication	<code>app.py</code> , <code>utils/auth.py</code>
SQLAlchemy	Database ORM	<code>models/user_model.py</code>
Pandas	Data processing	<code>main.py</code> , scrapers
python-docx	DOCX parsing	<code>app.py</code> (resume upload)
PyPDF2	PDF parsing	<code>app.py</code> (resume upload)

14. Application Flow

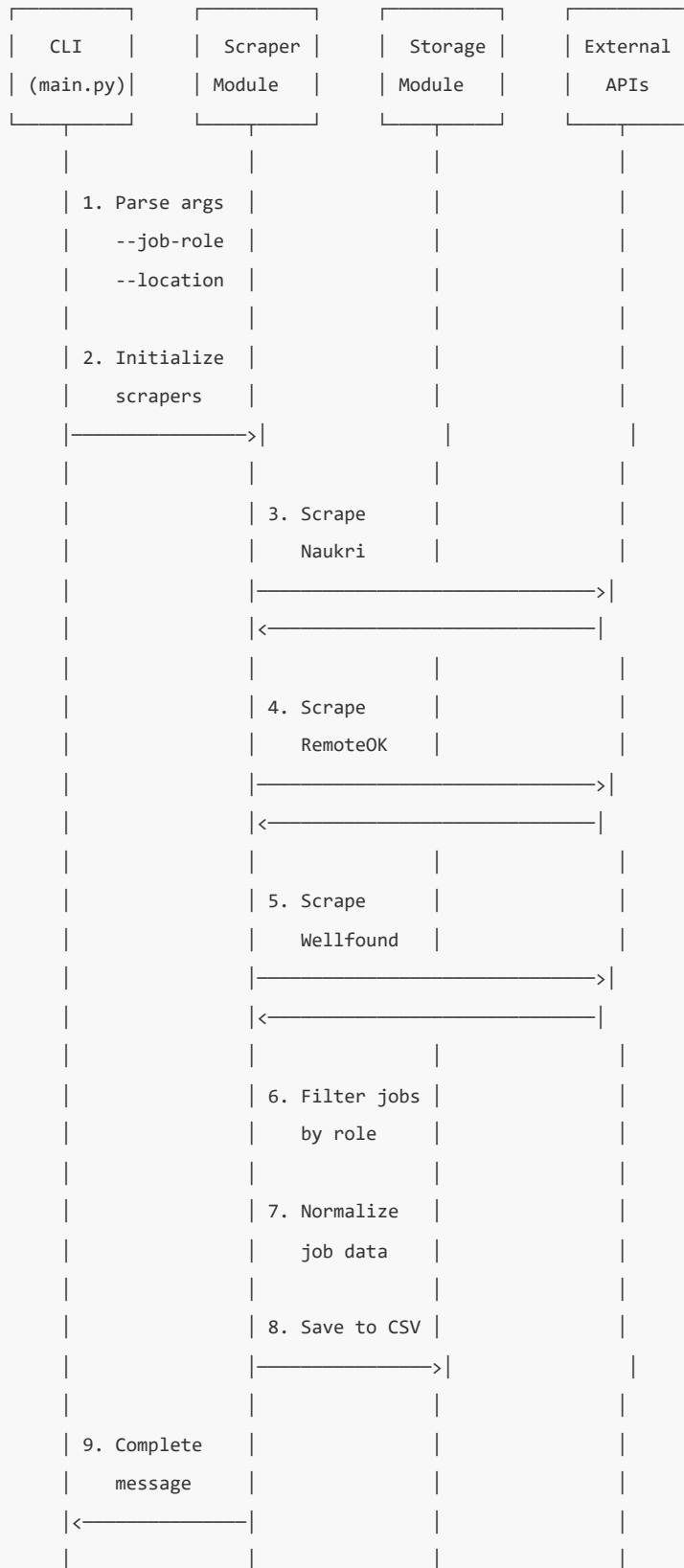
14.1 User Registration Flow



14.2 Resume Analysis Flow



14.3 Job Scraping Flow (CLI)



14.4 Complete User Journey

1. User visits homepage
 - ↳ GET /
 - ↳ Render index.html
2. User searches for "Python Developer"
 - ↳ POST /api/search
 - ↳ Return role guide, related roles, platform URLs
3. User views role guide
 - ↳ Display responsibilities, skills, salary, interview topics
4. User uploads resume
 - ↳ POST /api/analyze-resume
 - ↳ Call N8N workflow (or use built-in engine)
 - ↳ Return ATS score, suggestions, skill gap
5. User views skill gap analysis
 - ↳ POST /api/skill-gap
 - ↳ Return missing skills, learning roadmap
6. User practices interview questions
 - ↳ POST /api/interview-prep
 - ↳ Return role-specific questions
7. User views career insights
 - ↳ POST /api/career-insights
 - ↳ Return market data, salary ranges, top companies
8. User signs up
 - ↳ POST /api/auth/signup
 - ↳ Create account, profile, preferences, session
9. User saves jobs to dashboard
 - ↳ POST /api/dashboard/save-job
 - ↳ Store in database
10. User submits feedback
 - ↳ POST /api/feedback
 - ↳ Call N8N workflow for sentiment analysis
 - ↳ Save to database, send email if critical

15. Security & Deployment

15.1 Security Best Practices

Implemented

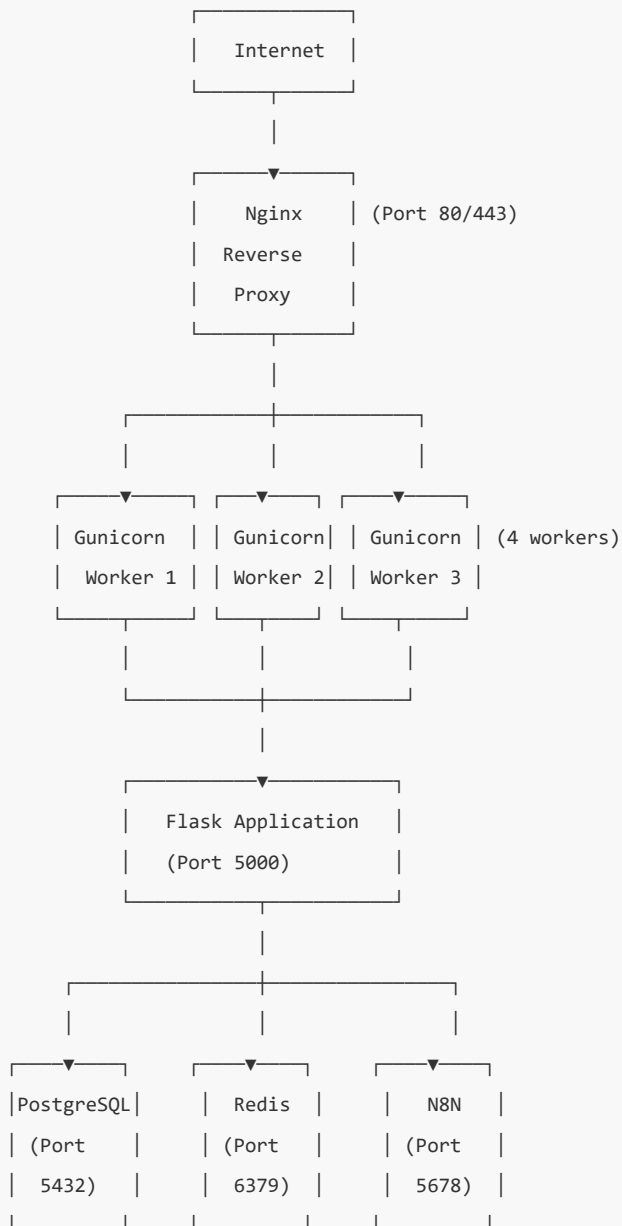
-  Password hashing (Werkzeug/pbkdf2)
-  Account lockout after failed attempts
-  Session token generation (cryptographically secure)
-  Input sanitization (XSS prevention)
-  SQL injection prevention (SQLAlchemy ORM)
-  CSRF protection (Flask-WTF ready)
-  Audit logging
-  Rate limiting (configurable)
-  Secure headers (via Nginx)
-  Environment variable configuration
-  No secrets in code

Recommended

-  Implement email verification
-  Add rate limiting middleware
-  Enable HTTPS/TLS in production
-  Implement JWT refresh tokens
-  Add CAPTCHA for signup/login
-  Implement password complexity meter
-  Add two-factor authentication (2FA)
-  Regular security audits
-  Dependency vulnerability scanning

15.2 Deployment Architecture

Production Stack



15.3 Docker Deployment

Build and Run

```
# Build images
docker-compose build

# Start all services
docker-compose up -d

# View logs
docker-compose logs -f app

# Stop services
docker-compose down

# Stop and remove volumes
docker-compose down -v
```

Services

Service	Image	Ports	Purpose
postgres	postgres:15-alpine	5432	Primary database
redis	redis:7-alpine	6379	Cache and sessions
n8n	n8nio/n8n:latest	5678	Workflow automation
app	Custom build	5000	Flask application
nginx	nginx:alpine	80, 443	Reverse proxy
prometheus	prom/prometheus:latest	9090	Metrics collection
grafana	grafana/grafana:latest	3000	Monitoring dashboards

15.4 Environment Setup

Development

```
# 1. Clone repository
git clone https://github.com/username/job-agent.git
cd job-agent

# 2. Create virtual environment
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate

# 3. Install dependencies
pip install -r requirements.txt

# 4. Copy environment file
cp .env.example .env

# 5. Edit .env with your values
nano .env

# 6. Initialize database
python -c "from app import app, db; app.app_context().push(); db.create_all()"

# 7. Run application
python app.py
```

Production

```
# 1. Set environment variables
export DATABASE_URL=postgresql://user:pass@localhost:5432/jobagent
export SECRET_KEY=your-secret-key-here
export GEMINI_API_KEY=your-gemini-key

# 2. Build Docker image
docker-compose build

# 3. Start services
docker-compose up -d

# 4. Verify services
docker-compose ps
docker-compose logs -f
```

15.5 Monitoring & Logging

Prometheus Metrics

- Endpoint: `http://localhost:9090`
- Metrics: Request count, response time, error rate
- Retention: 30 days

Grafana Dashboards

- Endpoint: `http://localhost:3000`
- Default credentials: admin/admin (change immediately)
- Dashboards: Application metrics, database performance, N8N workflow stats

Application Logs

```
# Log levels
LOG_LEVEL=INFO # DEBUG, INFO, WARNING, ERROR, CRITICAL

# Log format
%(asctime)s - %(name)s - %(levelname)s - %(message)s

# Log destinations
# - Console (stdout)
# - File (if LOG_FILE configured)
```

15.6 Backup Strategy

Database Backup

```
# PostgreSQL backup
docker exec jobagent-postgres pg_dump -U jobagent jobagent > backup.sql

# Restore
docker exec -i jobagent-postgres psql -U jobagent jobagent < backup.sql
```

File Backup

```
# Backup volumes
docker run --rm -v jobagent-postgres_data:/data -v $(pwd):/backup alpine tar cvf
/backup/postgres_backup.tar /data
docker run --rm -v jobagent-redis_data:/data -v $(pwd):/backup alpine tar cvf /backup/redis_backup.tar
/data
docker run --rm -v jobagent-n8n_data:/data -v $(pwd):/backup alpine tar cvf /backup/n8n_backup.tar /data
```

16. Developer Guide

16.1 Getting Started

Prerequisites

- Python 3.11+
- PostgreSQL 15+ (or SQLite for development)
- Redis 7+ (optional)
- Node.js 18+ (for frontend build tools)
- Docker & Docker Compose (for containerized deployment)

Installation Steps

1. Clone repository
2. Create virtual environment
3. Install dependencies
4. Configure environment variables
5. Initialize database
6. Run development server

16.2 Code Style

Python

- **Style Guide:** PEP 8
- **Line Length:** 100 characters
- **Indentation:** 4 spaces
- **Naming:** snake_case for functions/variables, PascalCase for classes

JavaScript

- **Style Guide:** ES6+
- **Format:** 2-space indentation
- **Naming:** camelCase for functions/variables, PascalCase for classes

16.3 Testing

Run Tests

```
# All tests
pytest

# With coverage
pytest --cov=.

# Specific test file
pytest tests/test_naukri_scraper.py

# Verbose output
pytest -v
```

Test Structure

```
tests/
├─ test_naukri_scraper.py      # Naukri scraper tests
├─ test_remoteok_scraper.py   # RemoteOK scraper tests
└─ test_wellfound_scraper.py  # Wellfound scraper tests
```

16.4 Adding New Features

Adding a New Scraper

1. Create new file in `scrapers/` directory
2. Inherit from `BaseScraper`
3. Implement `scrape()` method
4. Add to `main.py`
5. Write tests

Adding a New API Endpoint

1. Define route in `app.py`
2. Implement business logic

3. Add authentication if needed
4. Update frontend JavaScript
5. Document in API section

Adding a New N8N Workflow

1. Create workflow JSON in `n8n/workflows/`
2. Import in N8N UI
3. Configure credentials
4. Test webhook trigger
5. Document integration points

16.5 Debugging

Enable Debug Mode

```
# In app.py
app.run(debug=True, host='0.0.0.0', port=5000)
```

Logging

```
from utils.logger import get_logger
logger = get_logger()
logger.debug("Debug message")
logger.info("Info message")
logger.warning("Warning message")
logger.error("Error message")
```

Database Queries

```
# Enable SQL logging
app.config['SQLALCHEMY_ECHO'] = True

# Query analysis
from sqlalchemy import debug
```

17. Project Summary & Recommendations

17.1 Project Strengths

✅ Comprehensive Feature Set

- Multi-platform job scraping
- AI-powered resume analysis
- Interview preparation
- Career insights
- User management

✅ Robust Architecture

- Separation of concerns
- Modular design
- Scalable structure
- Docker containerization

✅ Security Best Practices

- Password hashing
- Account lockout
- Audit logging
- Input sanitization

✅ AI Integration

- Google Gemini for analysis
- N8N for automation
- Multiple AI providers supported

✅ Documentation

- Extensive inline comments
- README with usage examples
- API documentation
- Architecture diagrams

17.2 Areas for Improvement

⚠️ Testing

- Limited test coverage
- No integration tests
- Missing E2E tests

⚠️ Error Handling

- Inconsistent error responses
- Missing error boundaries
- Limited retry logic

⚠️ Performance

- No database query optimization
- Missing caching layer (Redis ready but not fully utilized)
- Synchronous AI API calls

⚠️ Monitoring

- Basic logging only
- No APM (Application Performance Monitoring)
- Missing health check endpoints

⚠️ Frontend

- No modern framework (React/Vue/Angular)
- Limited state management
- No component library

17.3 Recommended Enhancements

High Priority

1. Implement Comprehensive Testing

- Unit tests for all modules
- Integration tests for APIs
- E2E tests for critical flows
- Target: 80% code coverage

2. Add Rate Limiting

```
```python
from flask_limiter import Limiter
limiter = Limiter(app, key_func=get_remote_address)
```

```
@app.route('/api/auth/login', methods=['POST'])
```

```
@limiter.limit("10 per minute")
```

```
def api_auth_login():
```

```
...
```
```

3. Implement Caching Strategy

- Redis for session storage
- Cache API responses
- Cache database queries
- Implement cache invalidation

4. Add Health Check Endpoints

```
python @app.route('/health') def health_check(): return jsonify({ 'status': 'healthy',
'database': check_database(), 'redis': check_redis(), 'timestamp':
datetime.utcnow().isoformat() })
```

Medium Priority

5. Migrate Frontend to Modern Framework

- React or Vue.js
- Component library (Material-UI, Chakra UI)
- State management (Redux, Pinia)
- TypeScript for type safety

6. Implement JWT Authentication

- Replace session-based auth
- Add refresh tokens
- Implement token revocation

7. Add API Versioning

`/api/v1/auth/login /api/v2/auth/login`

8. Implement WebSocket Support

- Real-time notifications
- Live interview sessions
- Collaborative features

Low Priority

9. Add GraphQL API

- Flexible data fetching
- Reduce over-fetching
- Better mobile support

10. Implement Microservices

- Separate services for scraping, AI, user management
- Message queue (RabbitMQ, Kafka)
- Service mesh (Istio)

11. Add Machine Learning Pipeline

- Custom resume scoring model
- Job recommendation engine
- Salary prediction model

17.4 Performance Optimization

Database

- Add database indexes for frequently queried fields
- Implement connection pooling
- Use database read replicas
- Implement query caching

Application

- Implement async/await for I/O operations
- Use CDN for static assets
- Implement response compression
- Optimize image assets

Caching

- Redis for session storage
- Memcached for API responses
- Browser caching headers
- CDN caching

17.5 Security Enhancements

1. Implement HTTPS

- Obtain SSL certificates (Let's Encrypt)
- Configure Nginx for TLS
- Enable HSTS

2. Add Security Headers

```
nginx add_header X-Frame-Options "SAMEORIGIN"; add_header X-Content-Type-Options "nosniff";  
add_header X-XSS-Protection "1; mode=block"; add_header Content-Security-Policy "default-src  
'self'";
```

3. Implement CSP (Content Security Policy)

4. Add Rate Limiting

5. Implement IP Whitelisting for Admin

6. Add DDoS Protection

7. Regular Security Audits

17.6 Scalability Recommendations

Horizontal Scaling

- Load balancer (Nginx, HAProxy)
- Multiple application instances
- Database read replicas
- Redis cluster

Vertical Scaling

- Increase server resources
- Optimize database queries
- Implement caching
- Use CDN

Cloud Migration

- AWS/GCP/Azure deployment
- Managed databases (RDS, Cloud SQL)

- Managed caching (ElastiCache, Memorystore)
- Container orchestration (EKS, GKE, AKS)

17.7 Maintenance Plan

Daily

- Monitor error logs
- Check API rate limits
- Review feedback submissions

Weekly

- Review analytics data
- Update job scraping selectors
- Backup database

Monthly

- Update dependencies
- Security patches
- Performance review
- User feedback analysis

Quarterly

- Architecture review
 - Security audit
 - Feature prioritization
 - Technology stack evaluation
-

Appendix A: Quick Reference

A.1 Common Commands

```
# Development
python app.py                # Start Flask dev server
python main.py --job-role "Python Developer" # Scrape jobs

# Testing
pytest                       # Run tests
pytest --cov=.               # With coverage

# Docker
docker-compose up -d         # Start all services
docker-compose logs -f app   # View app logs
docker-compose down          # Stop services
docker-compose restart app    # Restart app

# Database
docker exec -it jobagent-postgres psql -U jobagent # Access PostgreSQL
```

A.2 Important URLs

| Service | URL | Credentials |
|-------------|-----------------------|-------------------------------|
| Application | http://localhost:5000 | - |
| N8N | http://localhost:5678 | admin/admin |
| Grafana | http://localhost:3000 | admin/admin |
| Prometheus | http://localhost:9090 | - |
| PostgreSQL | localhost:5432 | jobagent/secure_password_here |
| Redis | localhost:6379 | - |

A.3 Environment Checklist

- `[] DATABASE_URL` configured
- `[] SECRET_KEY` set (32+ characters)
- `[] GEMINI_API_KEY` added (for AI features)
- `[] SMTP_USER` and `[] SMTP_PASS` configured (for emails)
- `[] N8N_WEBHOOK_URL` set
- `[] REDIS_URL` configured (production)

- [] `FLASK_ENV=production` (production)
- [] `FLASK_DEBUG=False` (production)

End of Documentation

This document was auto-generated from comprehensive code analysis. For questions or clarifications, refer to inline code comments or contact the development team.
